

# Python Functions Notes

Topics 13 & 14: Local & Global Variables, and the global Keyword

## 13. Local & Global Variables

### 1. Variable Scope

Scope refers to the part of the program where a variable can be accessed. Python mainly has:

- **Local scope:** inside a function
- **Global scope:** outside a function

### 2. Local Variable

A variable created inside a function is called a local variable.

```
def greet():  
    message = "Hello"  
    print(message)
```

```
greet()
```

Output:  
Hello

Here, **message** is a local variable. It cannot normally be accessed outside the function:

```
def greet():  
    message = "Hello"
```

```
greet()
```

```
print(message)
```

Output:  
NameError

### Example

```
def calculate():  
    a = 10  
    b = 20  
    result = a + b  
  
    print(result)
```

```
calculate()
```

Here **a**, **b**, and **result** are local variables.

### 3. Global Variable

A variable created outside a function is called a global variable.

```
name = "Musafir"
```

```
def greet():  
    print(name)
```

```
greet()
```

Output:  
Musafir

The function can read the global variable.

#### 4. Local and Global Variables Together

```
name = "Alice"          # Global variable

def student():
    age = 20             # Local variable

    print(name)
    print(age)

student()
```

Output:  
Alice  
20

Here: **name** is Global, **age** is Local.

#### 5. Same Variable Name

A local variable can have the same name as a global variable.

```
name = "Alice"

def greet():
    name = "Bob"
    print(name)

greet()

print(name)

Output:
Bob
Alice
```

The local variable takes priority inside the function.

#### 6. Local vs Global

| Feature                   | Local Variable  | Global Variable              |
|---------------------------|-----------------|------------------------------|
| Created                   | Inside function | Outside function             |
| Scope                     | Function        | Entire program/module        |
| Accessed inside function  | Yes             | Yes                          |
| Accessed outside function | No              | Yes                          |
| Modified inside function  | Directly        | Requires global if assigning |

## 14. global Keyword

The **global** keyword tells Python that a variable inside a function refers to a global variable, rather than creating a new local variable.

### 1. Without global

```
count = 10

def update():
    count = 20
    print(count)

update()

print(count)

Output:
20
10
```

The **count** inside `update()` is a new local variable.

### 2. With global

```
count = 10

def update():
    global count
    count = 20

update()

print(count)

Output:
20
```

Now the global **count** was changed.

### 3. Example with Balance

```
balance = 1000

def deposit(amount):
    global balance
    balance += amount

deposit(500)

print(balance)

Output:
1500
```

The global keyword allows the function to modify **balance**.

### 4. Global Variable: Reading vs Modifying

You do not need **global** just to read a global variable.

```
x = 10

def display():
    print(x)

display()
```

But if you want to assign a new value to it inside the function:

```
x = 10

def update():
    global x
    x = 20

update()

print(x)
```

## 5. Important Rule

Reading a global variable  
|  
v  
global keyword NOT required

Modifying/assigning a global variable  
|  
v  
global keyword required

```
x = 10

def test():
    print(x)          # Reading -> no global needed

x = 10

def test():
    global x
    x = 50             # Modifying -> global needed
```

## Quick Revision

Local Variable  
-> Created inside a function.

Global Variable  
-> Created outside a function.

Scope  
-> Where a variable can be accessed.

global keyword  
-> Used to modify a global variable inside a function.

## Remember

```
x = 10
```

```
def example():  
    global x  
    x = 20
```

After calling **example()**: **x = 20**

*Next topic: Nested Functions.*